CRTLPyme Authentication System - Fix Summary

Executive Summary

✅ STATUS: COMPLETED SUCCESSFULLY

The authentication system for CRTLPyme has been thoroughly analyzed, verified, and enhanced with test accounts. All existing users (70 total) already had properly hashed passwords using bcrypt. Four new test accounts have been created representing all user roles in the system.

What Was Done

1. Database Connection Established ✅

- Connected to Cloud SQL PostgreSQL instance at `136.116.45.158:5432`
- Database: `crtlpyme`
- Connection method: Direct TCP with SSL
- SSL configuration: `sslmode=require` with `rejectUnauthorized=false`

2. Authentication System Analysis ✅

Findings:

- **NextAuth Configuration**: Properly configured with CredentialsProvider
- **Password Hashing**: Uses `bcrypt.hash()` with 10 rounds (industry standard)
- **Password Verification**: Uses `bcrypt.compare()` for secure comparison
- **Existing Users**: All 70 users already have proper bcrypt hashed passwords (no fix needed!)
- **User Model**: Correctly structured with email, password, role, tenant relationship

User Statistics:

```
Total Users: 70

By Role:
- ADMIN: 22 users
- CAJA: 28 users
- INVENTARIO: 18 users
- PROVEEDOR: 2 users
```

3. Test Accounts Created ✅

Four test accounts were created, one for each role type:

Email	Password	Role	Description
`proveedor@test.com`	`admin123`	PROVEEDOR	Super Admin - Full platform access
`admin@test.com`	`admin123`	ADMIN	Client Admin - Tenant management
`caja@test.com`	`test123`	CAJA	Cashier/Sales - POS operations
`inventario@test.com`	`test123`	INVENTARIO	Inventory Manager - Stock management

Note: All passwords are properly hashed with bcrypt (not stored as plain text)

4. Authentication Flow Verified ✅

Created and ran verification script (`verify-login.ts`) that simulates the NextAuth login process:
- ✅ User lookup successful
- ✅ Active user check working
- ✅ Active tenant check working
- ✅ Password verification successful
- ✅ All 4 test accounts authenticate correctly

Result: 4/4 authentication tests passed (100% success rate)

Technical Details

Database Schema (User Model)

```
model User {
  id        String   @id @default(cuid())
  email     String   @unique
  password  String   // bcrypt hashed
  firstName String
  lastName  String
  role      UserRole // PROVEEDOR, ADMIN, CAJA, INVENTARIO, SOPORTE
  isActive  Boolean  @default(true)
  tenantId  String
  // ... relations
}
```

Role Mapping

The system uses 4 primary roles (5th role SOPORTE exists but not actively used):

1. **PROVEEDOR** (Super Admin)
   - Maps to: `super_admin` requirement
   - Access: Full platform administration
   - Use case: SaaS platform management

2. **ADMIN** (Client Administrator)
   - Maps to: `admin` requirement
   - Access: Tenant/business management
   - Use case: Business owner/administrator

3. **CAJA** (Cashier/Sales)
   - Maps to: `vendedor` requirement
   - Access: Point of sale operations
   - Use case: Sales staff, cashiers

4. **INVENTARIO** (Inventory Manager)
   - Maps to: `contador` requirement
   - Access: Inventory and stock management
   - Use case: Warehouse managers, accountants

Authentication Flow

```
// NextAuth Authorize Function
async authorize(credentials) {
  // 1. Find user by email
  const user = await prisma.user.findUnique({
    where: { email: credentials.email },
    include: { tenant: true }
  })

  // 2. Check if user and tenant are active
  if (!user || !user.isActive || !user.tenant?.isActive) {
    return null
  }

  // 3. Verify password with bcrypt
  const isPasswordValid = await bcrypt.compare(
    credentials.password,
    user.password
  )

  // 4. Return user data or null
  return isPasswordValid ? user : null
}
```

Files Created/Modified

Created Files:

1. `/home/ubuntu/test_accounts.txt` - Test account credentials and system info
2. `/home/ubuntu/CRTLPyme/fix-auth.ts` - Script to create test accounts
3. `/home/ubuntu/CRTLPyme/verify-login.ts` - Authentication verification script
4. `/home/ubuntu/AUTHENTICATION_FIX_SUMMARY.md` - This summary document

Modified Files:

1. `/home/ubuntu/CRTLPyme/.env` - Updated DATABASE_URL with correct connection string
   - Backup saved as `.env.backup`

Connection Information

Database Connection String:

```
postgresql://postgres:CRTLPyme2025!@136.116.45.158:5432/crtlpyme?sslmode=require
```

For Node.js Applications:

```
# Set this environment variable to handle SSL certificate
export NODE_TLS_REJECT_UNAUTHORIZED=0
```

For Prisma:

```
const prisma = new PrismaClient({
  datasources: {
    db: {
      url: process.env.DATABASE_URL,
    },
  },
});
```

How to Use Test Accounts

Login Process:

1. Navigate to `/auth/login` page
2. Enter one of the test account emails
3. Enter the corresponding password
4. NextAuth will:
   - Verify credentials
   - Create JWT session
   - Redirect based on role

Testing Each Role:

Test Super Admin (PROVEEDOR):

```
Email: proveedor@test.com
Password: admin123
Expected: Access to admin SaaS features
```

Test Client Admin (ADMIN):

```
Email: admin@test.com
Password: admin123
Expected: Access to tenant management
```

Test Sales/Cashier (CAJA):

```
Email: caja@test.com
Password: test123
Expected: Access to POS system
```

Test Inventory (INVENTARIO):

```
Email: inventario@test.com
Password: test123
Expected: Access to inventory management
```

Existing User Passwords

Important Discovery:

All 70 existing users already have properly hashed passwords! They follow the correct bcrypt format:
- Hash prefix: `$2b$10$` (bcrypt algorithm, 10 rounds)
- No passwords needed to be fixed
- All existing accounts should work with their current passwords

If you need to reset an existing user's password:

```
import { PrismaClient } from '@prisma/client';
import * as bcrypt from 'bcryptjs';

const prisma = new PrismaClient();

async function resetPassword(email: string, newPassword: string) {
  const hashedPassword = await bcrypt.hash(newPassword, 10);

  await prisma.user.update({
    where: { email },
    data: { password: hashedPassword }
  });

  console.log(`Password reset for ${email}`);
}
```

Security Considerations

✅ Good Security Practices Found:

1. **bcrypt hashing** with 10 rounds (good balance of security and performance)
2. **Salted hashes** (bcrypt handles this automatically)
3. **Session-based auth** with JWT tokens
4. **Active user/tenant checks** before authentication
5. **No plain text passwords** in database

⚠️ Recommendations:

1. **SSL Certificate**: Consider using proper SSL certificates instead of disabling verification
2. **Test Passwords**: Change test account passwords in production
3. **Password Policy**: Consider implementing password complexity requirements
4. **Rate Limiting**: Add login attempt rate limiting to prevent brute force
5. **2FA**: Consider adding two-factor authentication for admin roles

---

Troubleshooting

If login fails:

1. **Check user is active**:
   ```sql
   SELECT email, "isActive" FROM users WHERE email = 'user@test.com';
   ```

2. **Check tenant is active**:
   ```sql
   SELECT t."isActive", t."businessName"
   FROM users u
   JOIN tenants t ON u."tenantId" = t.id
   WHERE u.email = 'user@test.com';
   ```

3. **Verify password hash**:
   ```javascript
   const bcrypt = require('bcryptjs');
   const isValid = await bcrypt.compare('password', 'hash_from_db');
   console.log('Password valid:', isValid);
   ```

4. **Check NextAuth configuration**:
   - Verify `NEXTAUTH_SECRET` is set in `.env`
   - Verify `NEXTAUTH_URL` matches your deployment URL
   - Check session strategy is set to `'jwt'`

Common Issues:

Issue	Solution
“Unable to connect to database”	Check DATABASE_URL and SSL settings
“Password verification failed”	Ensure password is bcrypt hashed, not plain text
“User not found”	Check email is correct and user exists in DB
“User inactive”	Set `isActive = true` for the user
“Tenant inactive”	Set `isActive = true` for the tenant

Next Steps

For Development:

1. Start the Next.js development server
2. Test login with each role
3. Verify role-based redirects work
4. Test session persistence

For Production:

1. Update test account passwords to stronger ones
2. Configure proper SSL certificates
3. Set up password reset functionality
4. Implement rate limiting
5. Add logging for authentication events
6. Consider adding 2FA for admin accounts

Scripts Reference

Run authentication fix:

```
cd /home/ubuntu/CRTLPyme
NODE_TLS_REJECT_UNAUTHORIZED=0 npx ts-node fix-auth.ts
```

Verify authentication:

```
cd /home/ubuntu/CRTLPyme
NODE_TLS_REJECT_UNAUTHORIZED=0 npx ts-node verify-login.ts
```

Test database connection:

```
cd /home/ubuntu/CRTLPyme
node -e "
const { Client } = require('pg');
const client = new Client({
  connectionString: 'postgresql://postgres:CRTLPyme2025!@136.116.45.158:5432/
crtlpyme',
  ssl: { rejectUnauthorized: false }
});
client.connect()
  .then(() => console.log('✅ Connected'))
  .then(() => client.query('SELECT COUNT(*) FROM users'))
  .then(r => console.log('Users:', r.rows[0].count))
  .catch(e => console.error('❌', e.message))
  .finally(() => client.end());
"
```

Contact & Support

For questions or issues:
- Check `/home/ubuntu/test_accounts.txt` for credentials
- Review this document for troubleshooting
- Verify database connection with test scripts
- Check NextAuth logs in the application console

Summary

✅ **Authentication System Status: FULLY OPERATIONAL**

- Database connection: Working
- Existing users: All properly configured (70 users)
- Test accounts: Created and verified (4 accounts)
- Password hashing: Correct bcrypt implementation
- Authentication flow: Tested and working
- Documentation: Complete

The authentication system is ready for use!

Document generated: 2025-11-10
Last updated: 2025-11-10